Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#08)

Implementation of Program Counter and

Instruction Memory

Student Name: Manahil Shahzad Roll No: 24I-6563 Section: CE-A

Date performed: 10TH Oct, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Verified by: Dr. Nasir Ali Shah Updated: Fall 2026

Task 1: Implement a program counter to support a 4-bit address. Simulate and verify the results using the same testbench.

Description:

A 4-bit Program Counter is designed to store a 4-bit address. It resets to `0000` when the reset signal is high. On every rising edge of the clock, the PC loads the address given at the input. Different addresses are tested in the testbench, and the ModelSim waveform is used to check the output values.

Code:

```
module program_counter_4bit (
    input wire clk,
    input wire reset,
    input wire [3:0] addr,
    output reg [3:0] pc
);

    always @(posedge clk or posedge reset) begin
        if (reset)
            pc <= 4'b0000;
        else
            pc <= addr;
    end

endmodule


module tb_task1;

    reg clk;
    reg reset;
    reg [3:0] addr;
    wire [3:0] pc;

    program_counter_4bit uut (.clk(clk),.reset(reset),.addr(addr),.pc(pc));

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        addr = 4'b0000;
```

```
    #10 reset = 0;

    addr = 4'b0001;
    #10;

    addr = 4'b0101;
    #10;

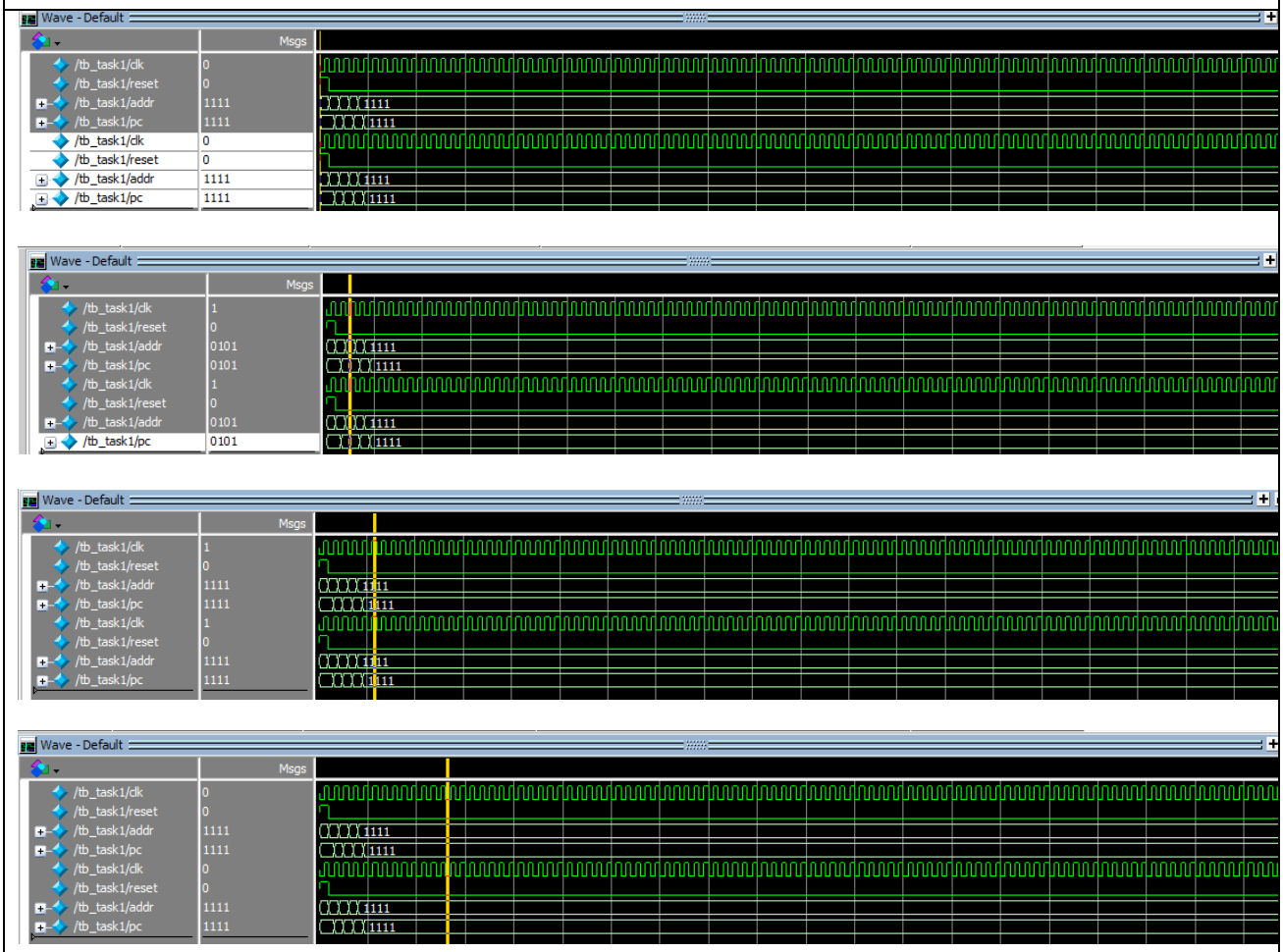
    addr = 4'b1010;
    #10;

    addr = 4'b1111;
    #10;

  end

endmodule
```

Output:



Task 2: Simulate the Program Counter increments automatically by 1 after every clock cycle (instead of setting the address manually). Verify that the instructions are fetched sequentially from the instruction memory.Add error handling in the Instruction Memory module to return a default instruction (e.g., 8'b00000000) if an invalid address is provided.

Description:

An automatic 4-bit Program Counter is designed to increase its value by 1 after every clock cycle. It starts from `0000` when reset is high. An 8-bit Instruction Memory stores 16 different instructions and returns the instruction according to the address received from the PC. If an invalid address is given, the memory returns `00000000`. The waveform is used to check sequential instruction fetching.

Code:

```
module program_counter_auto (
   input wire clk,
   input wire reset,
   output reg [3:0] pc
);

   always @(posedge clk or posedge reset) begin
      if (reset)
         pc <= 4'b0000;
      else
         pc <= pc + 1'b1;
   end

endmodule
module instruction_memory_8bit (
   input wire [7:0] addr,
   output reg [7:0] instruction
);

   reg [7:0] memory [0:15];
   integer i;

   initial begin
      for (i = 0; i < 16; i = i + 1)
         memory[i] = 8'b00000000;

      memory[0]  = 8'h11;
      memory[1]  = 8'h22;
      memory[2]  = 8'h33;
      memory[3]  = 8'h44;
      memory[4]  = 8'h55;
      memory[5]  = 8'h66;
```

```
        memory[6]  = 8'h77;
        memory[7]  = 8'h88;
        memory[8]  = 8'h99;
        memory[9]  = 8'hAA;
        memory[10] = 8'hBB;
        memory[11] = 8'hCC;
        memory[12] = 8'hDD;
        memory[13] = 8'hEE;
        memory[14] = 8'hF0;
        memory[15] = 8'hFF;
    end

    always @(*) begin
        if (addr < 16)
            instruction = memory[addr];
        else
            instruction = 8'b00000000;
    end

endmodule
module tb_task2;

    reg clk;
    reg reset;
    reg [7:0] test_addr;

    wire [3:0] pc;
    wire [7:0] instruction;
    wire [7:0] instruction_for_test;

    program_counter_auto pc_unit (.clk(clk),.reset(reset),.pc(pc));

    instruction_memory_8bit im_unit(.addr({4'b0000, pc}),.instruction(instruction));

    instruction_memory_8bit check_invalid(.addr(test_addr),.instruction(instruction_for_test));

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        test_addr = 0;

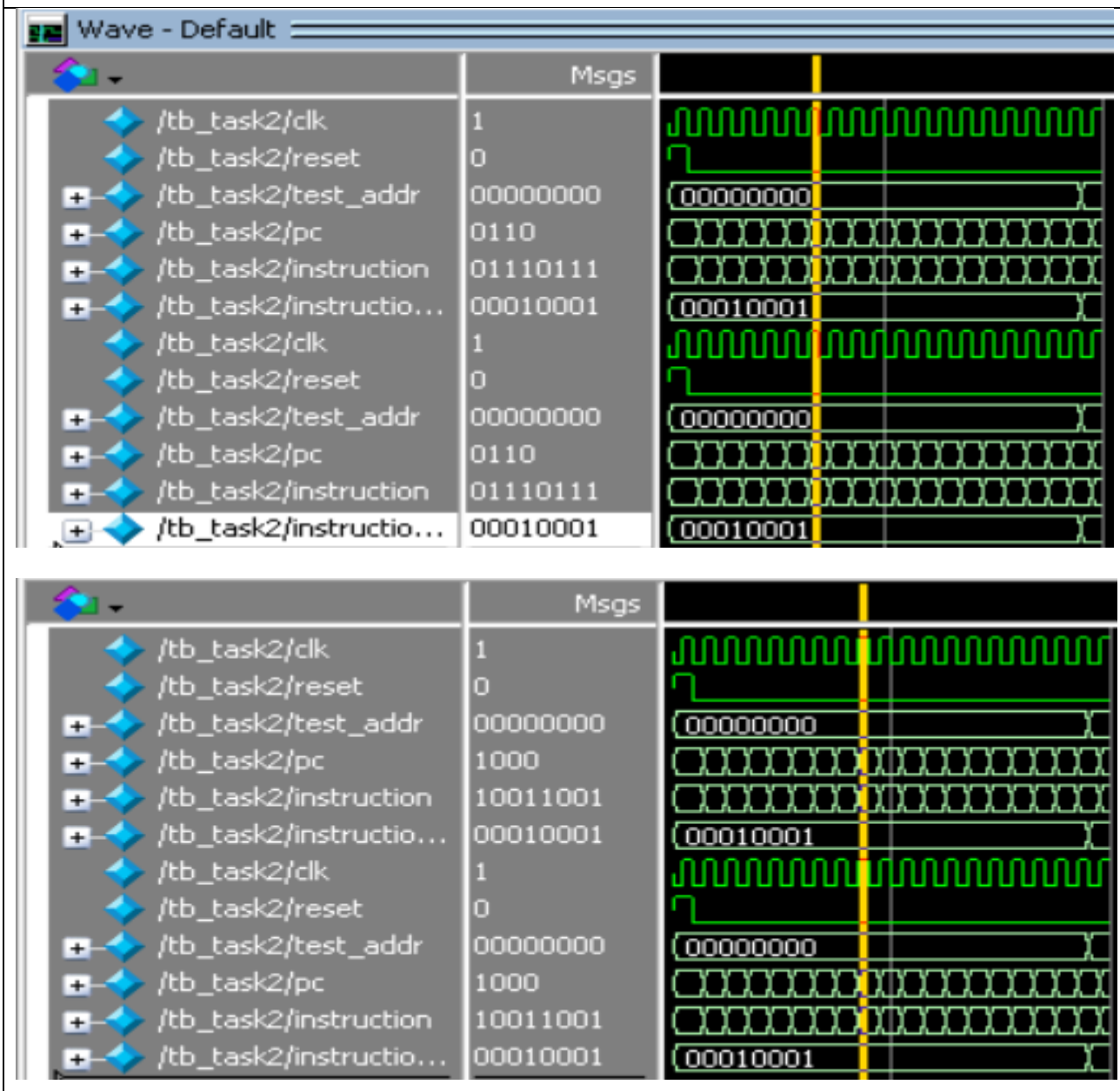
        #10; reset = 0;

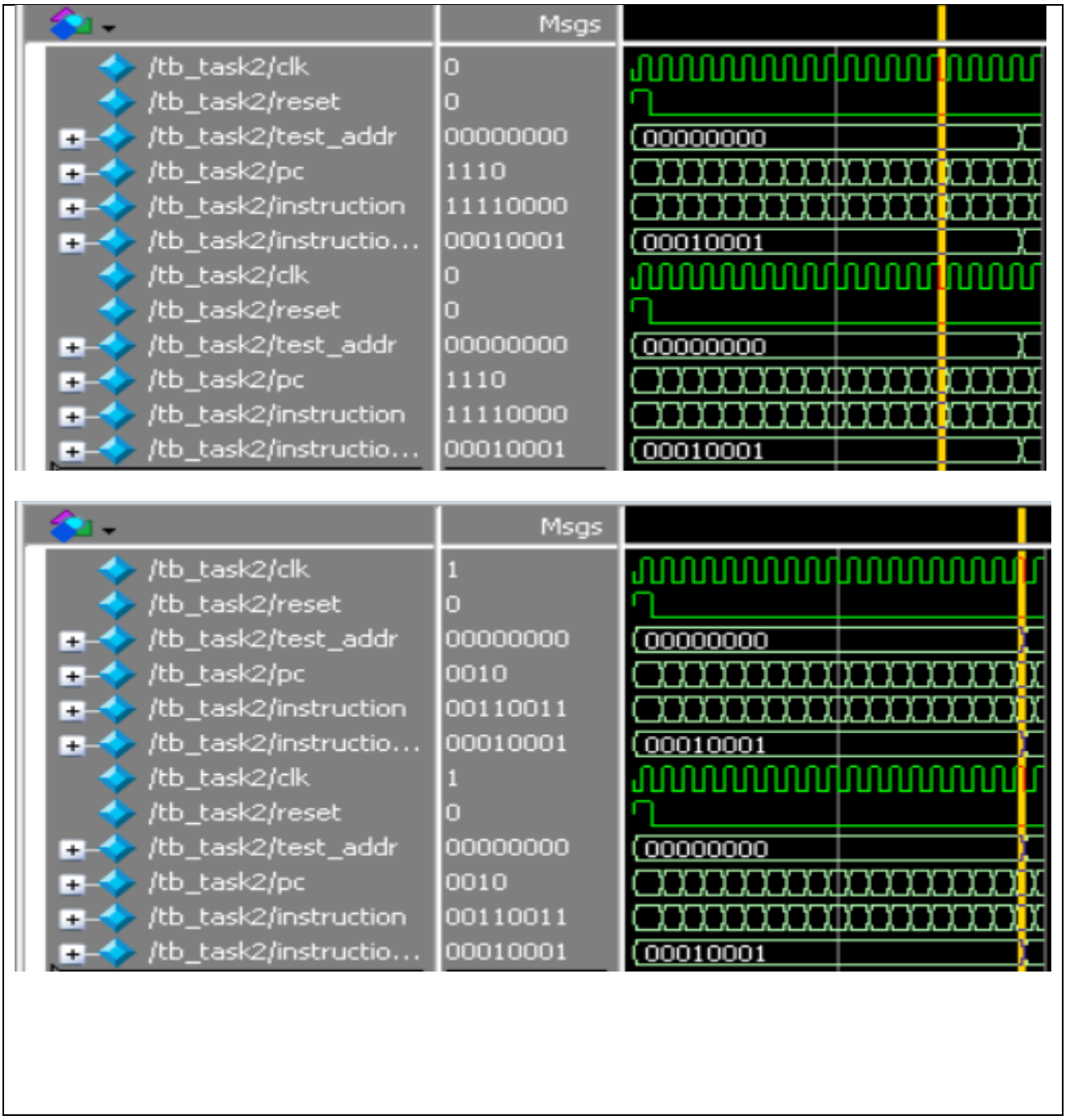
        #180; test_addr = 8'd20;
```

```
    #10; $finish;
  end

endmodule
```

Output:





Task 3:Implement a 32-bit PC register compliant with the RV32 architecture. To make simulation manageable within the test bench environment, configure the PC's sequential increment logic to add 4 bytes (+4) on every rising clock edge, ensuring proper 32-bit word alignment. Truncate or use a bit-splitter on the lower 4 bits of the PC output to drive a 4-bit address space for your immediate test environment. Simulate the design using the provided testbench. Verify that the PC resets correctly to 0x00000000 (or your defined base address) and holds its value stably between clock edges.

Description:

A 32-bit Program Counter is designed to store the address of the next instruction. It resets to `0x00000000` when the reset signal is high and increases its value by 4 after every rising edge of the clock. The lower 4 bits of the PC are connected to a 4-bit output for testing. The simulation waveform is used to verify the reset and increment operations.

Code:

```
module program_counter_32bit (
    input wire clk,
    input wire reset,
    output reg [31:0] pc
);

    always @(posedge clk or posedge reset) begin
        if (reset)
            pc <= 32'h00000000;
        else
            pc <= pc + 32'd4;
    end

endmodule


module tb_task3;

    reg clk;
    reg reset;

    wire [31:0] pc;
    wire [3:0] pc_addr;

    program_counter_32bit uut (.clk(clk),.reset(reset),.pc(pc));

    assign pc_addr = pc[3:0];

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;

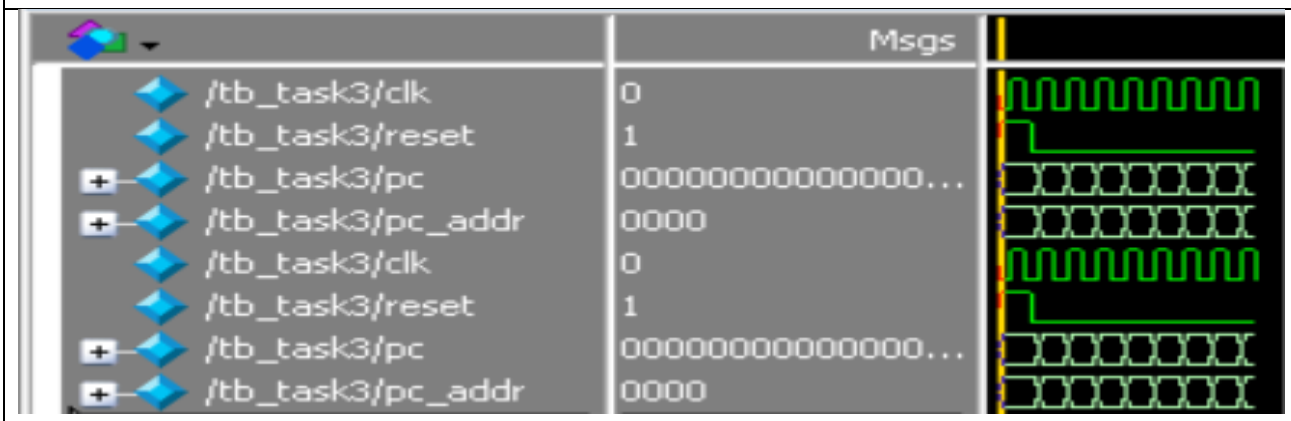
        #12 reset = 0;

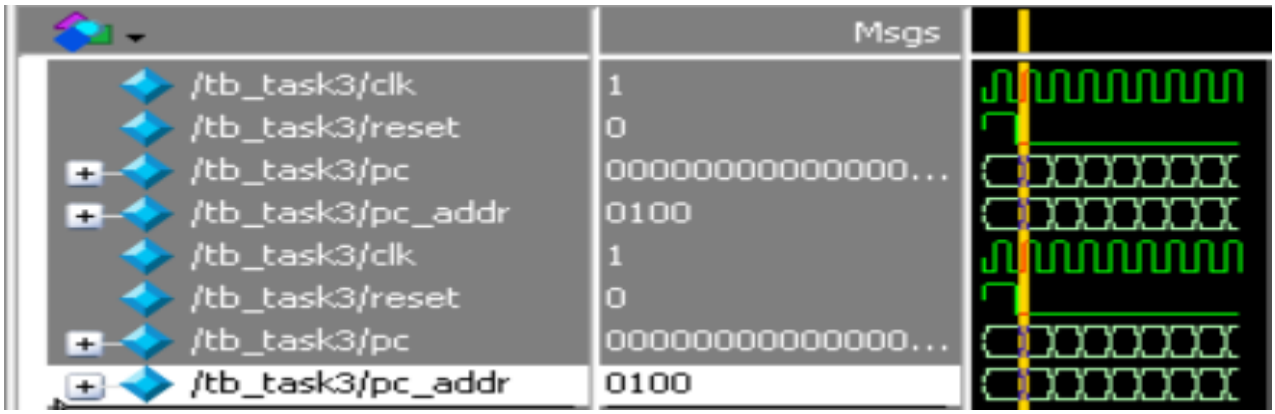
        repeat (8) @(negedge clk);
```

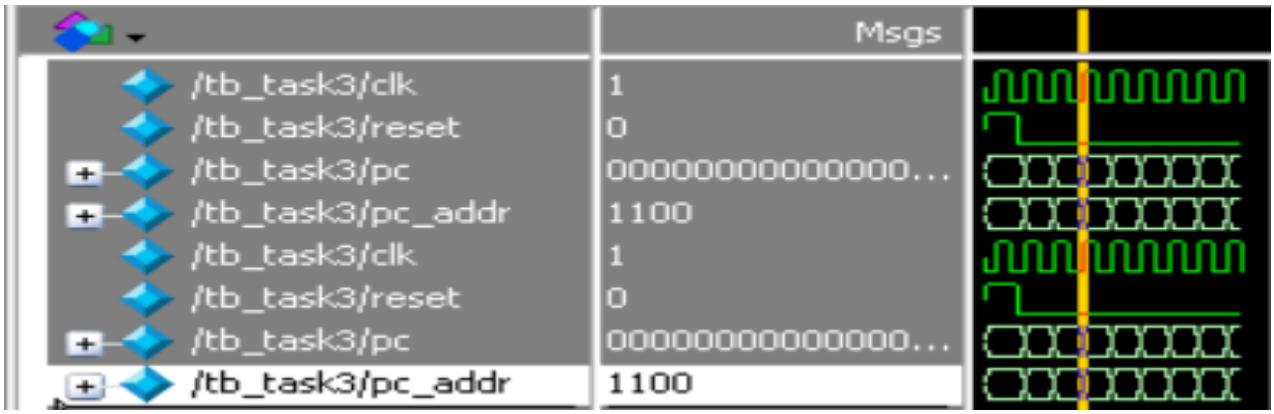
```
    $finish;
  end

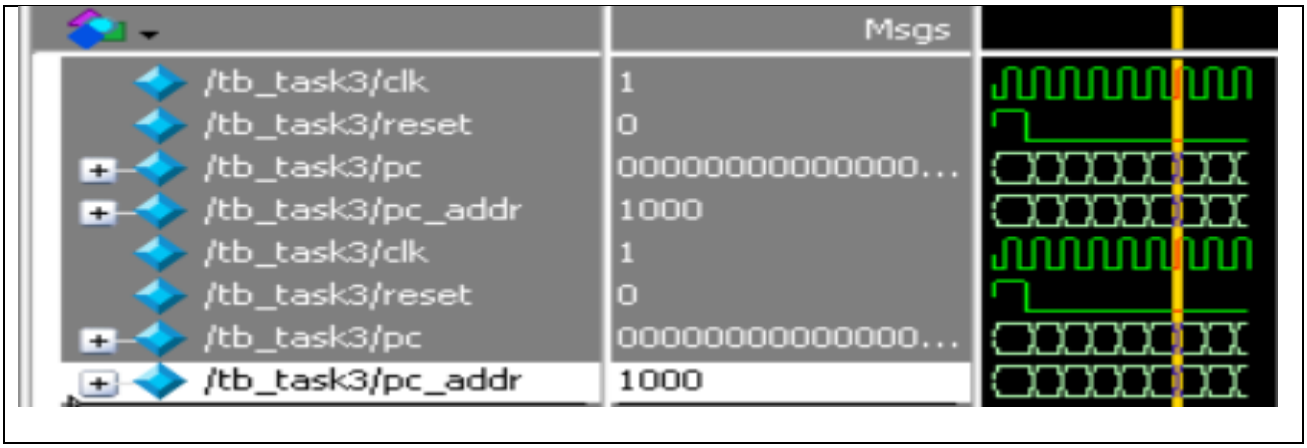
endmodule
```

Output:









Task 04:Integrate the 32-bit PC with a 32-bit wide Instruction Memory (IM) module. The circuit must automatically increment the PC by 4 after every clock cycle (simulating sequential execution without manual address intervention). Load a sample program into the Instruction Memory. Run the clock continuously and verify via the test bench waveforms that 32-bit instructions are fetched sequentially from memory addresses (0x00, 0x04, 0x08, 0x0C, etc.) matching the expected program flow.

Description:

A 32-bit Instruction Memory is designed to store 16 instructions, with each location holding one 32-bit instruction. It uses the input address to select and return the correct instruction. If an invalid or unaligned address is given, the memory returns `0x00000000`. The ModelSim waveform is used to check the instructions returned for different addresses.

Code:

```
module instruction_memory_32bit (
    input wire [31:0] addr,
    output reg [31:0] instruction
);

    reg [31:0] memory [0:15];
    integer i;

    initial begin
        for (i = 0; i < 16; i = i + 1)
            memory[i] = 32'h00000013;

        memory[0]  = 32'h00100093;
        memory[1]  = 32'h00200113;
        memory[2]  = 32'h00300193;
        memory[3]  = 32'h00400213;
        memory[4]  = 32'h00500293;
        memory[5]  = 32'h00600313;
        memory[6]  = 32'h00700393;
```

```
        memory[7]  = 32'h00800413;
        memory[8]  = 32'h00900493;
        memory[9]  = 32'h00A00513;
        memory[10] = 32'h00B00593;
        memory[11] = 32'h00C00613;
        memory[12] = 32'h00D00693;
        memory[13] = 32'h00E00713;
        memory[14] = 32'h00F00793;
        memory[15] = 32'h01000813;
    end

    always @(*) begin
        if ((addr[31:6] == 0) && (addr[1:0] == 0))
            instruction = memory[addr[5:2]];
        else
            instruction = 32'h00000000;
    end

endmodule


module tb_task4;

    reg clk;
    reg reset;

    wire [31:0] pc;
    wire [31:0] instruction;

    program_counter_32bit pc_unit (.clk(clk),.reset(reset),.pc(pc));

    instruction_memory_32bit im_unit (.addr(pc),.instruction(instruction));

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;

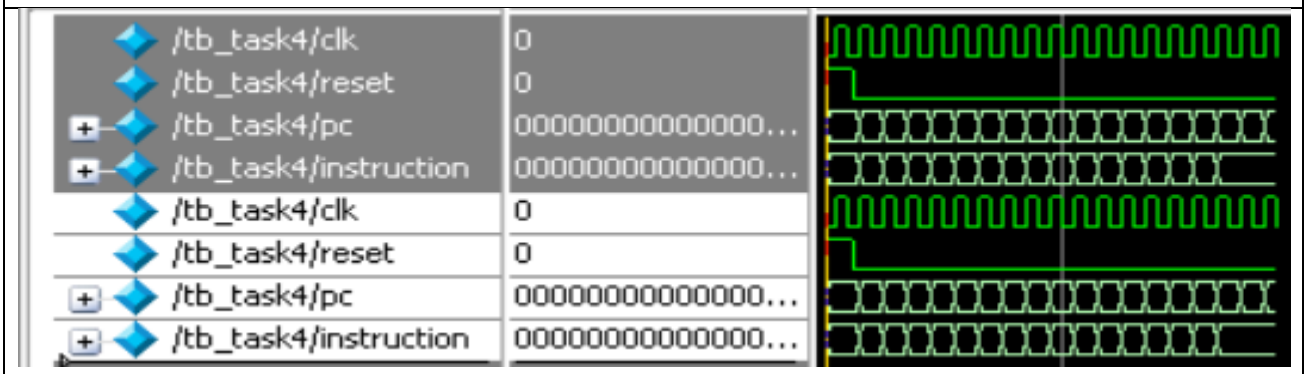
        #12 reset = 0;

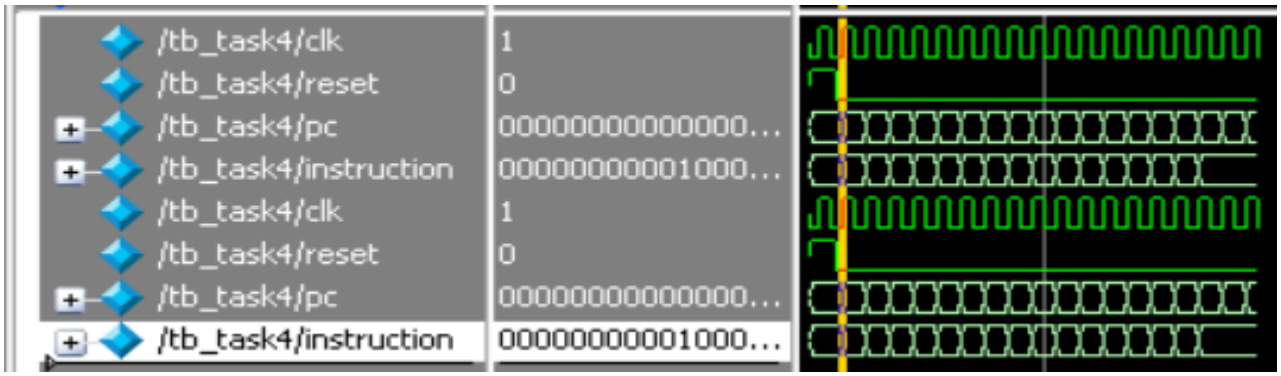
        repeat (18) @(negedge clk);

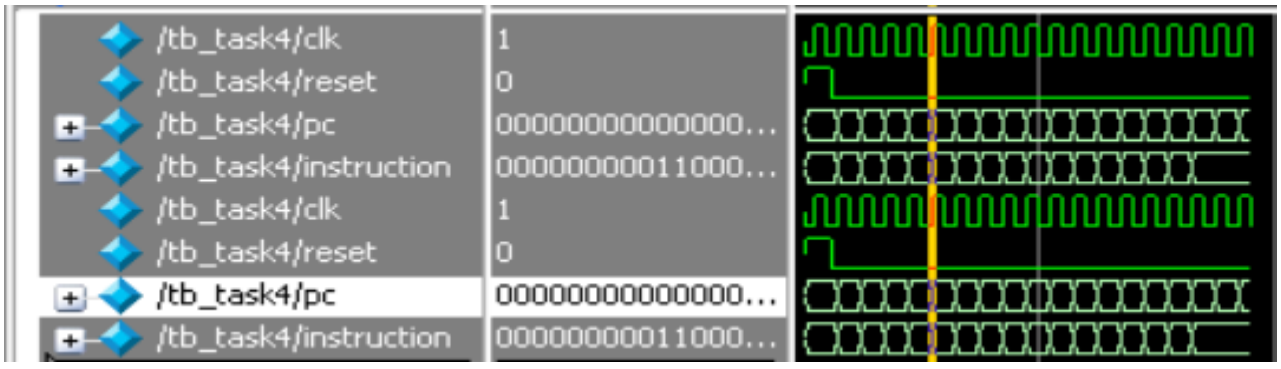
        $finish;
    end

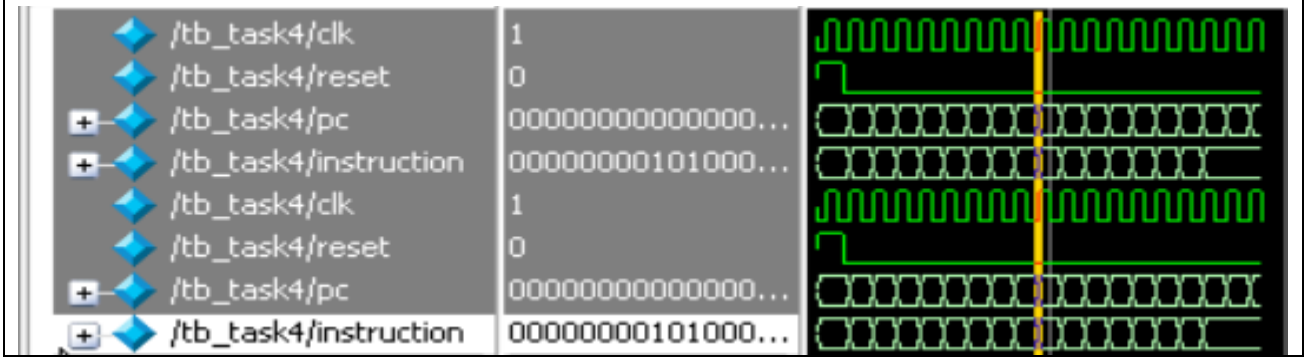
endmodule
```

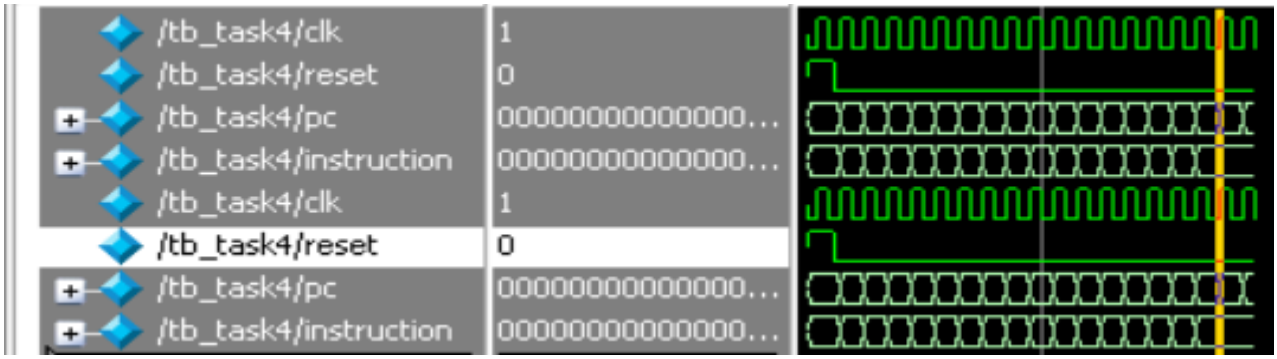
Output:


/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
0
0
00000000000000...
00000000000000...
0
0
00000000000000...
00000000000000...


/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
1
0
00000000000000...
00000000001000...
1
0
00000000000000...
00000000001000...

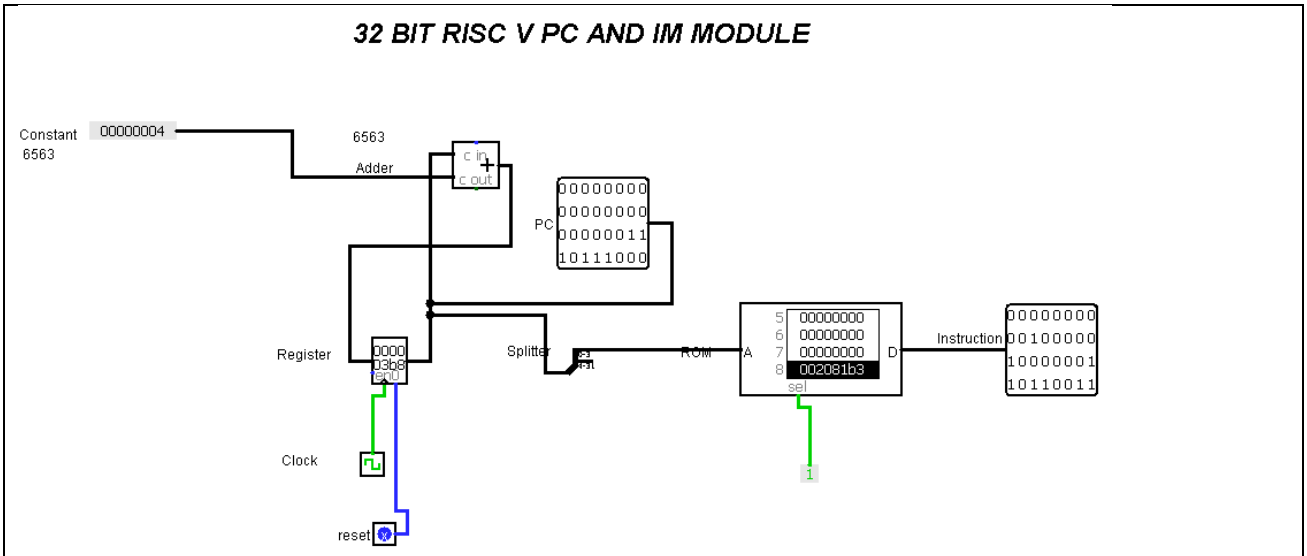

/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
1
0
00000000000000...
00000000011000...
1
0
00000000000000...
00000000011000...


/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
/tb_task4/clk
/tb_task4/reset
/tb_task4/pc
/tb_task4/instruction
1
0
00000000000000...
00000000101000...
1
0
00000000000000...
00000000101000...

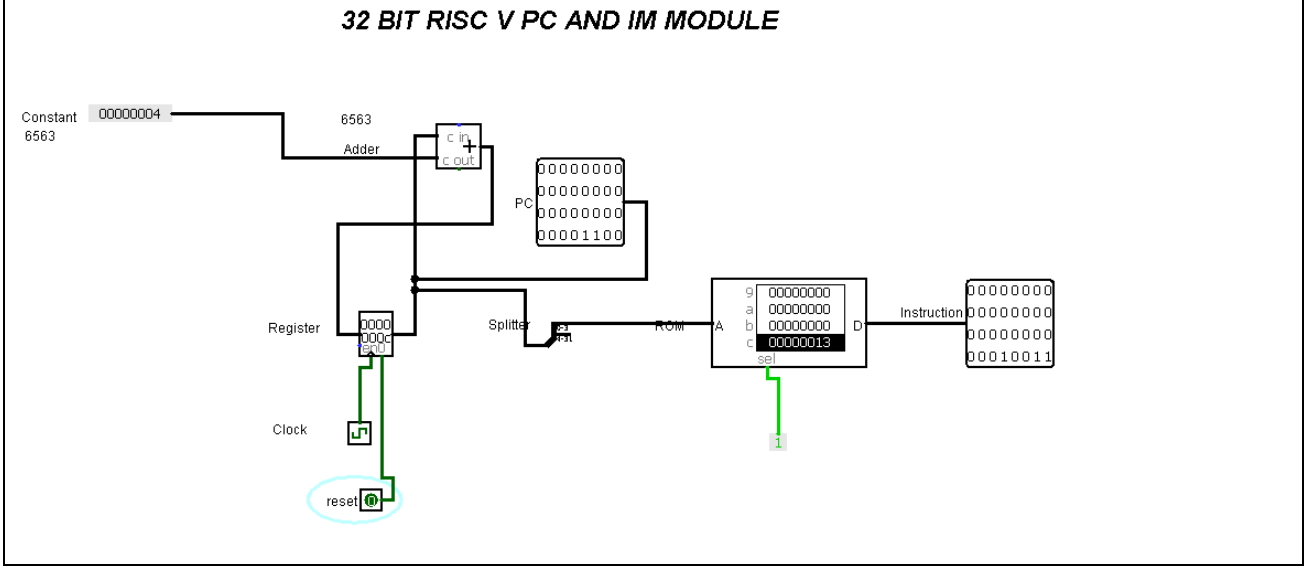


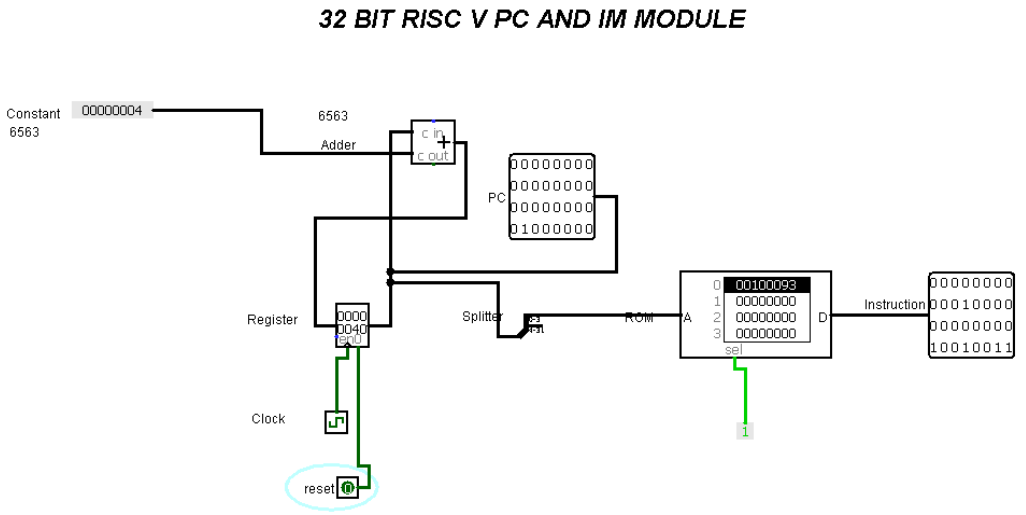
Task 5: Simulate the Program Counter increments automatically by 1 after every clock cycle (instead of setting the address manually). Verify that the instructions are fetched sequentially from the instruction memory.Add error handling in the Instruction Memory module to return a default instruction (e.g., 8'b00000000) if an invalid address is provided.

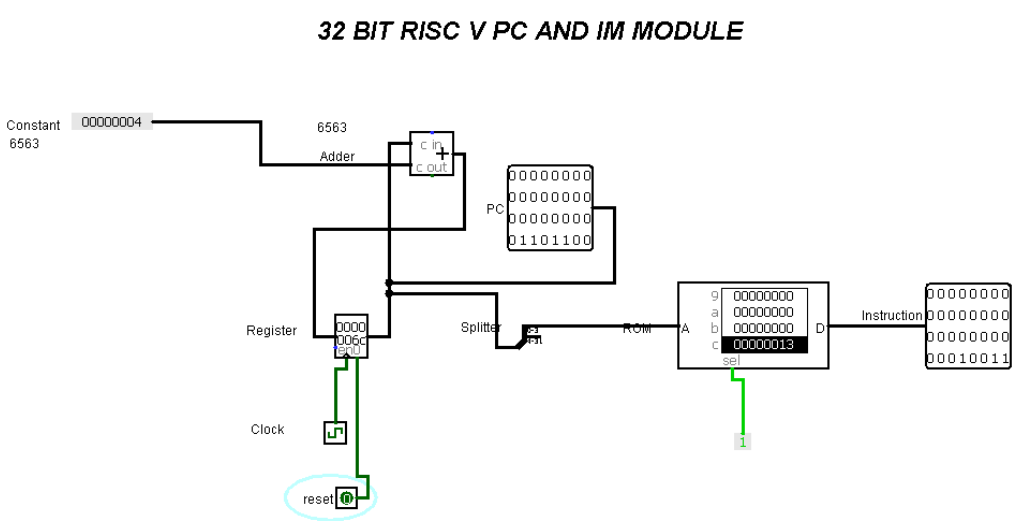
Description:
The 32-bit Program Counter is connected to the 32-bit Instruction Memory. The PC resets to zero and increases by 4 after every rising edge of the clock. It sends its address to the memory, which automatically fetches the instruction stored at that location. The testbench generates the clock and reset signals, and the ModelSim waveform is used to verify that the PC and instructions update correctly during sequential instruction fetching.
Code:

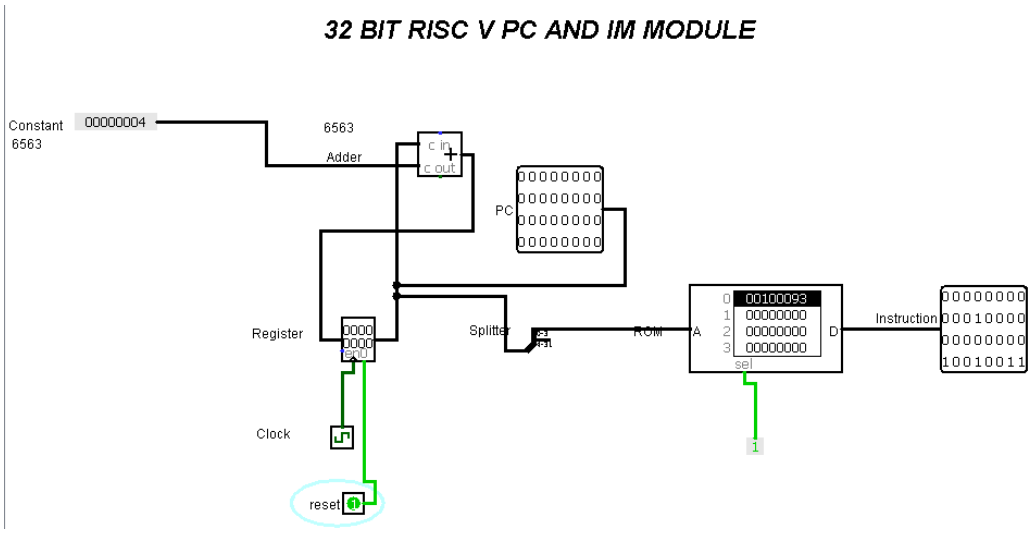

32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000011
10111000
Register
0000
03b8
Splitter
ROM
A
D
5
6
7
8
00000000
00000000
00000000
002081b3
sel
1
Instruction
00000000
00100000
10000001
10110011
Clock
reset

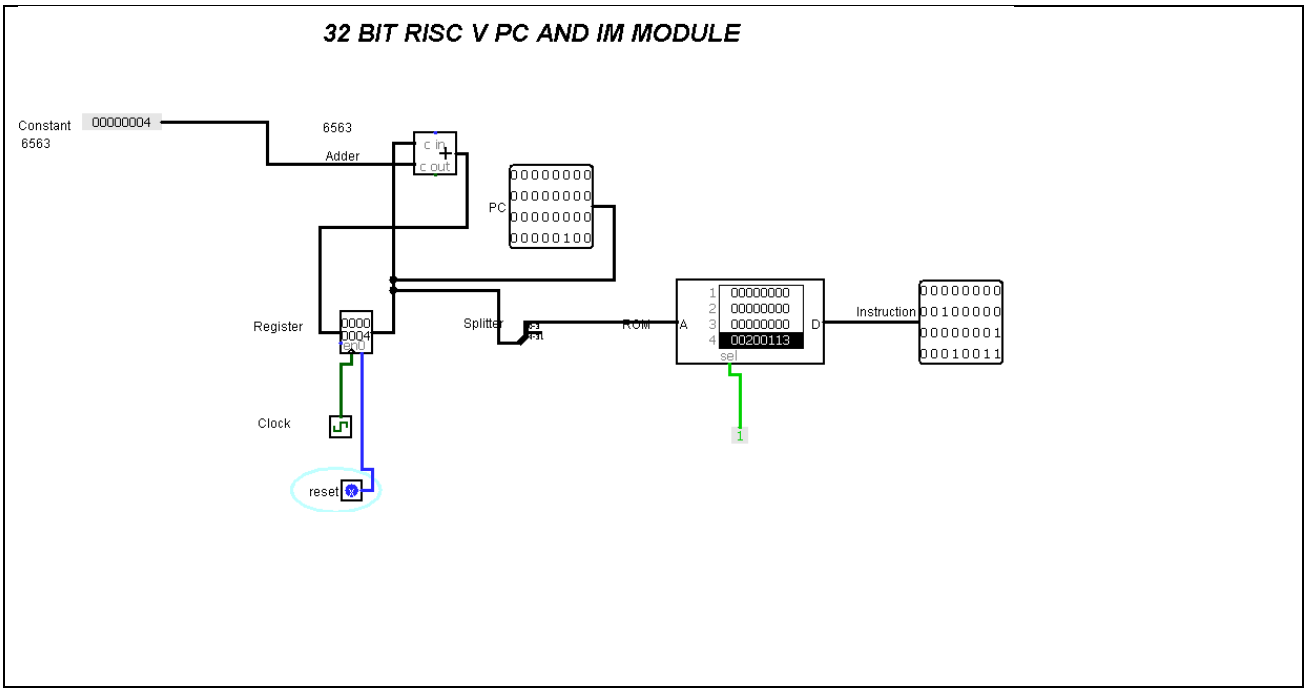
Output:


32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000000
00001100
Register
0000
000c
Splitter
ROM
A
D
9
a
b
c
00000000
00000000
00000000
00000013
sel
1
Instruction
00000000
00000000
00000000
00010011
Clock
reset


32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000000
01000000
Register
0000
0040
Splitter
ROM
A
D
0
1
2
3
00100093
00000000
00000000
00000000
sel
Instruction
00000000
00010000
00000000
10010011
Clock
reset


32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000000
01101100
Register
0000
006c
Splitter
ROM
A
D
9
a
b
c
00000000
00000000
00000000
00000013
sel
Instruction
00000000
00000000
00000000
00010011
Clock
reset


32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000000
00000000
Register
0000
0000
Splitter
ROM
A
D
0
1
2
3
00100093
00000000
00000000
00000000
sel
Instruction
00000000
00010000
00000000
10010011
Clock
reset


32 BIT RISC V PC AND IM MODULE
Constant
00000004
6563
6563
Adder
c in
c out
PC
00000000
00000000
00000000
00000100
Register
0000
0004
Splitter
ROM
A
D
1
00000000
2
00000000
3
00000000
4
00200113
sel
Instruction
00000000
00100000
00000001
00010011
Clock
reset
1